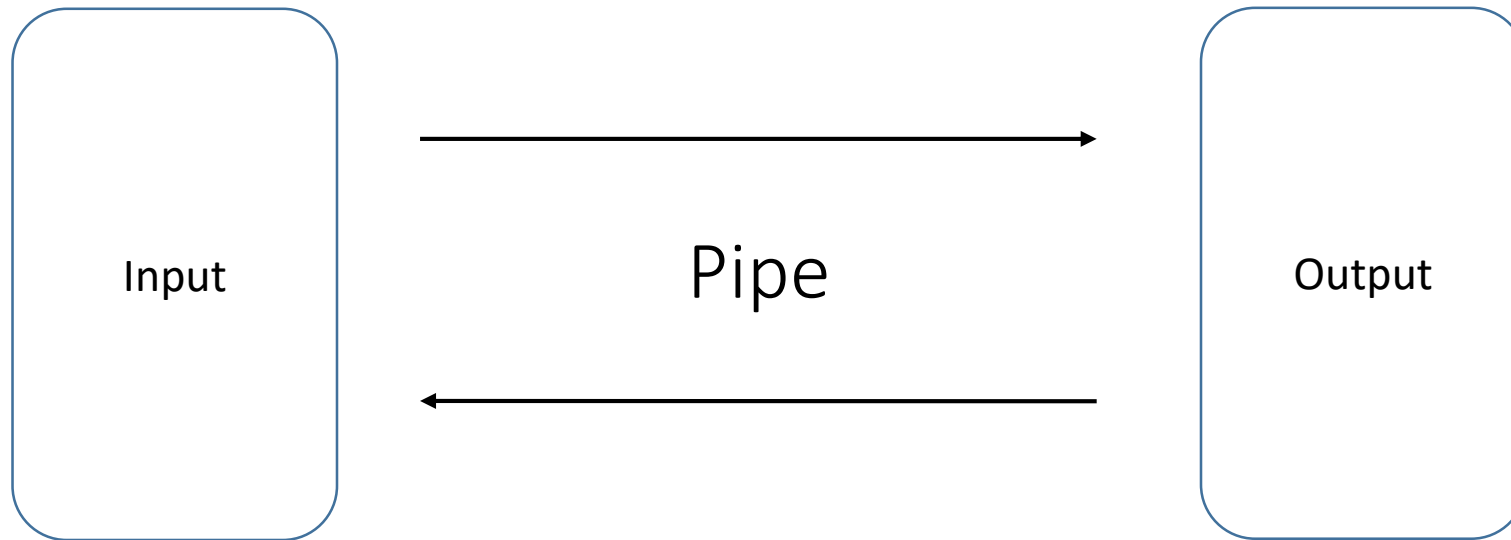


A thread-safe pipe implementation

Evangelos Zampras | Evangelos Balamotis

ECE, University of Thessaly



Αποφυγή των Race Conditions

Εντοπίσαμε race conditions (ταυτόχρονη εγγραφή και ανάγνωση από/προς σωλήνα), που ουσιαστικά αφορούν την προστασία του count variable μέσα στο circular buffer.

Για να τα αποφύγουμε εισάγουμε μεταβλητές read, write και turn για να δηλώσουμε πότε μια υπορουτίνα θέλει να διαβάσει, να γράψει στον κάθε ένα σωλήνα και πότε είναι η σειρά του read και του write.

Η υλοποίηση είναι παρόμοια με τους αλγορίθμους Dekker και Peterson που αποτελούν αποδεδειγμένες λύσεις στο πρόβλημα του mutual exclusion.

Συναρτήσεις βιβλιοθήκης

Κατασκευάσαμε τις εξής συναρτήσεις ως “API” της βιβλιοθήκης:

Συνάρτηση	Τιμή επιστροφής	Χρήση
pipe_read()	1: success, 0: pipe closed, -1: pipe not found	Διάβασμα 1 byte από αγωγό
pipe_write()	1: success, -1: pipe not found	Γράψιμο 1 byte σε αγωγό
pipe_open()	<id>: success, -1: pipe not created	Άνοιγμα νέου αγωγού
pipe_writeDone()	1: success, -1: pipe not found	Κλείσιμο αγωγού για γράψιμο
pipe_init()	void	Δημιουργία εσωτερικής δομής pipe system
pipe_destroy()	void	Εκκαθάριση αγωγού και αποδέσμευση μνήμης

Υλοποίηση βιβλιοθήκης

Η βιβλιοθήκη μας λειτουργεί εσωτερικά με βάση πίνακες που αποθηκεύουν δεδομένα για τον κάθε αγωγό. Τα δεδομένα αυτά είναι:

- Αν ο αγωγός υπάρχει (Το id του αν υπάρχει, -1 αν δεν υπάρχει)
- Αν ο αγωγός είναι ανοιχτός (1 ή 0)
- Αν κάποιο thread επιθυμεί γράψιμο
- Αν κάποιο thread επιθυμεί διάβασμα
- Η τιμή της μεταβλητής turn (σειρά του thread για διάβασμα ή γράψιμο, με βάση τον αλγόριθμο Peterson)
- Οι διευθύνσεις των εσωτερικών buffers ενός αγωγού, αν αυτός έχει δημιουργηθεί
- Το συνολικό πλήθος αγωγών που έχουμε δημιουργήσει

Η υλοποίηση λειτουργεί σωστά με αυτά τα δεδομένα καθώς έχουμε τον περιορισμό του specification ότι μόνο ένα thread γράφει και μόνο ένα thread διαβάζει έναν αγωγό κάθε στιγμή.

Testbenches

Έχουμε δημιουργήσει δύο testbenches με διαφορετικούς στόχους, για τη δοκιμή της λειτουργικότητας της βιβλιοθήκης αγωγών. (\$ make για να δημιουργηθούν και τα δύο)

./main (Source: main.c): Testbench που περιγράφεται στο cp_assignments_1.pdf

Δημιουργία δύο αγωγών 64 byte, «αποστολή» ενός αρχείου (input1) από ένα thread μέσω ενός αγωγού (p1) και «παραλαβή» του αρχείου από το άλλο thread. Το thread 2 αποθηκεύει το αρχείο σε αντίγραφο (output1).

Στη συνέχεια, μεταφορά του ίδιου αρχείου προς την ακριβώς αντίθετη κατεύθυνση μέσω ενός δεύτερου αγωγού (p2) και αποθήκευση ενός δεύτερου αντιγράφου (output2).

./main2 (Source: main2.c): Testbench με 5 αγωγούς που λειτουργούν ταυτόχρονα.

Δημιουργία 10 threads, τα 5 γράφουν σε 5 αγωγούς και τα υπόλοιπα διαβάζουν από τους αντίστοιχους αγωγούς. Ως αποτέλεσμα, 5 αρχεία input «αντιγράφονται» σε 5 αρχεία output.

Σημ.: Τα αρχεία εισόδου και εξόδου σε κάθε περίπτωση πρέπει να υπάρχουν ήδη πριν την έναρξη του testbench.

Testbenches

- Τα testbenches εκτυπώνουν παραπάνω πληροφορίες για τη λειτουργία των threads (πότε ένα thread ξεκινά/σταματά, κ.λπ.)
- Η λειτουργία δοκιμάστηκε με αρχεία διαφόρων μεγεθών (από μερικά KB έως και 5.3 MB), ώστε να διαπιστωθεί ότι οι πληροφορίες γράφονται σωστά μέσω των pipes.
- Πρέπει να κληθεί η `pipe_init()` πριν τη δημιουργία αγωγών!

A thread-safe pipe implementation

Evangelos Zampras | Evangelos Balamotis

ECE, University of Thessaly

Thanks for your time!